



Filesystem optimizations





Filesystem impact on performance

Tuning the filesystem is usually one of the first things we work on in boot time projects.

- ▶ Different filesystems can have different initialization and mount times. In particular, the type of filesystem for the root filesystem directly impacts boot time.
- ▶ Different filesystems can exhibit different read, write and access time performance, according to the type of filesystem activity and to the type of files in the system.



Different filesystem for different storage types

- ▶ Block storage (including memory cards, eMMC)
 - ext2, ext4
 - xfs, btrfs
 - f2fs
 - SquashFS, EROFS
- ▶ Raw flash storage
 - JFFS2
 - YAFFS2
 - UBIFS
 - ubiblock + (SquashFS or EROFS)

See our embedded Linux training materials for full details: <https://bootlin.com/doc/training/embedded-linux/>



Block filesystems For block storage

- ▶ ext4: pretty good read and write performance.
- ▶ xfs: can be good in some read or write scenarios as well.
- ▶ btrfs, f2fs: can achieve good read and write performance, taking advantage of the characteristics of flash-based block devices. However, btrfs is slow to initialize (see benchmarks later).
- ▶ SquashFS: very good mount time and read performance, for read-only partitions. Gives priority to compression rate vs performance.
- ▶ EROFS: newer read-only file system for block storage. Gives priority to read performance vs compression rate.



JFFS2 For raw flash storage

- ▶ Mount time depending on filesystem size: the kernel has to scan the whole storage at mount time, to read which block belongs to each file.
- ▶ Need to use the `CONFIG_JFFS2_SUMMARY` kernel option to store such information in flash. This dramatically reduces mount time.
- ▶ Benchmark on ARM: from 16 s to 0.8 s for a 128 MB partition.
- ▶ Rather poor read and write performance, compared to YAFFS2 and UBIFS.
- ▶ JFFS2 only makes sense on small storage space, where UBI would have too much overhead.



YAFFS2 For raw flash storage

- ▶ Good mount time
- ▶ Good read and write performance
- ▶ Drawbacks: no compression, not in the mainline Linux kernel



UBIFS For raw flash storage, on top of the UBI layer

▶ Advantages:

- Good read and write performance (similar to YAFFS2)
- Other advantages: better for wear leveling (can operate on the whole UBI space, not only within a single partition).

▶ Drawbacks:

- Not appropriate for small partitions (too much metadata overhead). Use JFFS2 or YAFFS2 instead.
- Not so good mount time, because of the time needed to initialize UBI (*UBI Attach*: at boot time or running `ubi_attach` in user space).
- Addressed by *UBI Fastmap*, introduced in Linux 3.7. See next slides.



How UBI Fastmap works

- ▶ *UBI Attach*: needs to read UBI metadata by scanning all erase blocks. Time proportional to the storage size.
- ▶ *UBI Fastmap* stores such information in a few flash blocks (typically at UBI detach time during system shutdown) and finds it there at boot time.
- ▶ This makes *UBI Attach* time constant.
- ▶ If *Fastmap* information is invalid (unclean system shutdown, for example), it falls back to scanning (slower, but correct, and *Fastmap* will work again during the next boot).
- ▶ Details: ELCE 2012 presentation from Thomas Gleixner: https://elinux.org/images/a/ab/UBI_Fastmap.pdf



Using UBI Fastmap

- ▶ Compile your kernel with `CONFIG_MTD_UBI_FASTMAP`
- ▶ Boot your system at least once with the `ubi.fm_autoconvert=1` kernel parameter.
- ▶ Reboot your system in a clean way
- ▶ You can now remove `ubi.fm_autoconvert=1`



UBI Fastmap benchmark

- ▶ Measured on the Microchip SAMA5D3 Xplained board (ARM), Linux 3.10
- ▶ UBI space: 216 MB
- ▶ Root filesystem: 80 MB used (Yocto)
- ▶ Average results:

	Attach time	Diff	Total time
Without <i>UBI Fastmap</i>	968 ms		
With <i>UBI Fastmap</i>	238 ms	−731 ms	−665 ms

- ▶ Expect to save more with bigger UBI spaces!

Note: total boot time reduction a bit lower probably because of other kernel threads executing during the attach process.



ubiblock + (SquashFS or EROFS) For raw flash storage

- ▶ *ubiblock*: read-only block device on top of UBI (`CONFIG_MTD_UBI_BLOCK`).
- ▶ Allows to put SquashFS or EROFS on a UBI volume.
- ▶ Expecting great boot time and read performance. Great for read-only root filesystems.
- ▶ Benchmarks not available yet.



Finding the best filesystem

- ▶ Raw flash storage: UBIFS with `CONFIG_UBI_FASTMAP` is probably the best solution.
- ▶ Block storage: SquashFS best solution for root filesystems which can be read-only. Btrfs and f2fs probably the best solutions for read/write filesystems.
- ▶ Fortunately, changing filesystem types is quite cheap, and completely transparent for applications. Just try several filesystem options, as see which one works best for you!
- ▶ Do not focus only on boot time. For systems in which read and write performance matters, we recommend to use separate root filesystem (for quick boot time) and data partitions (for good runtime performance).